

Test Task for QA Engineer — API Automation

Overview

This task evaluates your ability to **design and structure** an API test framework using **PHP + Codeception** against a REST API contract.

Important: The API described in this document does **not** exist as a running service. You will not be given a base URL, a Docker image, or any test environment. We are not interested in whether your tests pass against a live system — we are interested in **how you abstract, organize, and express tests against a contract you have only seen on paper.**

Treat the contract below as if it were a real API your team will integrate against next sprint. Design the test code accordingly: clean abstraction over the HTTP boundary, reusable request/response models, a clear path from “spec change” to “test change,” and a strategy for plugging this into CI once a real environment exists.

Test Task — API Automation Design

Part 1 — Tech Assignment

- Read the API contract for the **Media Buyers** resource (Section: **API Contract** below).
- Select the test scenarios you consider most important for automation across both endpoints.
- Design and implement an automated test suite using **PHP (Codeception)** with the **REST** and **PhpBrowser** modules (or a justified alternative). The suite must:
 - Encode every acceptance criterion as one or more tests.
 - Use factories or builders for request payloads (no hard-coded JSON inside test methods).
 - Validate every successful response against a JSON Schema.
 - Be parameterized where the contract calls for it (boundary cases, validation messages).
- Implement **at least 8 tests** in total — covering both the GET and POST endpoints, with positive and negative paths.
- **The tests do not need to pass against a real server**, and you are not required to build any test infrastructure to make them runnable. The focus of the evaluation is the test code itself, not the surrounding harness. If you *want* to demonstrate executability — for example by stubbing the HTTP layer, spinning up a small mock server (WireMock, Prism, the PHP built-in server, etc.), or marking tests **@skip** with full assertions written — you are welcome to, but it is not a graded deliverable.

- Treat the contract as the source of truth. If you have to make assumptions where the contract is silent, document them briefly in the README rather than guessing in code.

Part 2 — Written Evaluation

In addition to the automation task, please provide a short written answer — one to two pages total for all five questions:

1. Which scenarios did you select to automate, and **why** those over the alternatives? What did you intentionally leave out, and why?
2. Walk through the **abstractions** you introduced (HTTP client wrapper, factories, schema-validation helper, etc.) and explain what each one buys you when the suite grows from 8 tests to 80.
3. How would you approach **contract-drift detection** so that a backend change to this API automatically triggers test updates and review? Include tooling and process.
4. If you owned the QA process for this API, which **tools (including AI)** would you use for test generation, maintenance, flakiness detection, and reporting — and why?
5. What is the most challenging situation you have faced in API or end-to-end test automation, and how did you resolve it?

Part 3 — Deliverables

1. **A public GitHub repository** containing:
 - **Automated Tests Solution** — PHP/Codeception tests implementing the assignment.
 - **README.md** describing the project. There is no requirement to make the suite runnable, so the README is about **explaining what you built**, not about installation steps. It should cover:
 - A short overview of the Codeception API testing setup you would use for this kind of project — suite layout, the modules you'd enable (REST, PhpBrowser, Asserts, etc.).
 - How the repository is organized and what each part is for.
 - A short written note covering:
 - * Why you selected these scenarios.
 - * The abstractions you introduced and what they buy at scale.
 - * What additional improvements would make the project more scalable and maintainable (data setup/teardown, parallelization, CI integration, schema-versioning, contract testing, etc.).
 - * Any assumptions you made where the contract is silent.
 - JSON Schemas, request fixtures, and any helpers you wrote.
2. **Written Evaluation Document** (.docx, .pdf, or .md) — containing your answers to Part 2.

API Contract

Base URL

{{BASE_URL}}/api

There is no live server. Treat {{BASE_URL}} as a placeholder that would be set per environment in a real deployment (e.g. via `.env`). Your suite should be configured *as if* a real environment exists, with the URL resolved from configuration — not hard-coded in tests.

Conventions

- Content negotiation: requests must include **Content-Type: application/json** and **Accept: application/json**.
- All responses are JSON.
- Successful responses are wrapped in a top-level **data** object or array. Validation errors are wrapped in a top-level **errors** array.

The MediaBuyer resource

A single Media Buyer represents an internal employee who runs paid-traffic campaigns. Its fields:

Field	Type	Description
id	integer	Server-generated primary key. Read-only.
mbId	string	Business identifier (numeric string). Must be unique.
initials	string	Two uppercase letters used as a label, e.g. "SJ".
name	string	Full display name, between 2 and 30 characters.
email	string	RFC-compliant email address.
slackUserId	string	Slack workspace user ID, between 1 and 32 characters.
active	integer	1 if the buyer is active, 0 if inactive.

Endpoint 1 — GET /api/mediabuyers

Returns the full list of media buyers.

Request

GET /api/mediabuyers
Accept: application/json

- **Path parameters:** none.
- **Query parameters:** none.
- **Request body:** none.

Response — 200 OK

```
{
  "data": [
    {
      "id": 1,
      "mbId": "4534",
      "initials": "SJ",
      "name": "Sarah Jones",
      "email": "sarah.jones@example.com",
      "slackUserId": "U0690MMT3KJ",
      "active": 1
    }
  ]
}
```

Acceptance criteria

#	Criterion
G1	Returns HTTP 200 with Content-Type: application/json.
G2	Response body conforms to get-media-buyers-schema.json.
G3	The data field is always an array, even when the system has zero media buyers (returns {"data": []}, not 404 or null).
G4	Every item in data includes all required fields: id, mbId, initials, name, email, slackUserId, active.
G5	email values are syntactically valid email addresses.
G6	active is always 0 or 1 (integers, not booleans, not strings).
G7	id values are unique across the response.

Endpoint 2 — POST /api/mediabuyers

Creates a single media buyer.

Request

POST /api/mediabuyers
Content-Type: application/json
Accept: application/json

Request body:

```
{
  "mbId": "9001",
  "initials": "TM",
  "name": "Test Media Buyer",
  "email": "test.media.buyer@example.com",
  "slackUserId": "U05AZ3DQBBKK",
  "active": true
}
```

Field validation rules

Field	Required?	Rules
mbId	Yes	Non-empty string of digits representing a positive integer.
initials	No	If provided, must be exactly 2 characters long.
name	Yes	Length between 2 and 30 characters (inclusive).
email	Yes	Must match a strict, RFC-compliant email pattern.
slackUserId	No	If provided, length between 1 and 32 characters.
active	Yes	Boolean (true or false).

Response — 200 OK (success)

```
{
  "data": {
    "id": 42,
    "mbId": "9001",
    "initials": "TM",
    "name": "Test Media Buyer",
    "email": "test.media.buyer@example.com",
    "slackUserId": "U05AZ3DQBBKK",
    "active": 1
  }
}
```

Response — 400 Bad Request (validation error)

```
{
  "errors": [
```

```

    { "detail": "This field is missing: [name]" },
    { "detail": "The email not-an-email is not a valid email." }
  ]
}

```

Acceptance criteria

#	Criterion
P1	A valid request returns HTTP 200, a <code>Content-Type: application/json</code> header, and a body conforming to <code>post-media-buyer-schema.json</code> .
P2	The response <code>data.id</code> is a server-generated positive integer; the request never supplies it.
P3	The created media buyer is returned with the same <code>mbId</code> , <code>initials</code> , <code>name</code> , <code>email</code> , and <code>slackUserId</code> as the request.
P4	A request with <code>active: true</code> results in <code>data.active === 1</code> ; a request with <code>active: false</code> results in <code>data.active === 0</code> .
P5	Submitting a request that omits any required field (<code>mbId</code> , <code>name</code> , <code>email</code> , <code>active</code>) returns HTTP 400 and an <code>errors</code> array that names each missing field.
P6	Submitting <code>email: "not-an-email"</code> returns HTTP 400 and an error message that mentions the invalid email.
P7	Submitting <code>initials: "TOO LONG"</code> (more than 2 characters) returns HTTP 400 with the message <code>The initials must be exactly 2 characters long</code> .
P8	Submitting <code>name</code> shorter than 2 characters or longer than 30 returns HTTP 400 with a length-related error.
P9	Submitting <code>mbId: "abc"</code> (not a positive integer string) returns HTTP 400.
P10	Submitting <code>active: "yes"</code> (not a boolean) returns HTTP 400.
P11	Creating two media buyers with the same <code>mbId</code> returns an error on the second request (uniqueness). The candidate may treat the exact status code (400 or 409) as an open question and document the assumption.

JSON Schemas

tests/schemas/get-media-buyers-schema.json

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "data": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "id": { "type": "integer" },
          "mbId": { "type": "string" },
          "initials": { "type": "string", "minLength": 2, "maxLength": 2 },
          "name": { "type": "string", "minLength": 2, "maxLength": 30 },
          "email": { "type": "string", "format": "email" },
          "slackUserId": { "type": "string" },
          "active": { "type": "integer", "enum": [0, 1] }
        },
        "required": ["id", "mbId", "initials", "name", "email", "slackUserId", "active"]
      }
    }
  },
  "required": ["data"]
}
```

tests/schemas/post-media-buyer-schema.json

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "data": {
      "type": "object",
      "properties": {
        "id": { "type": "integer" },
        "mbId": { "type": "string" },
        "initials": { "type": "string", "minLength": 2, "maxLength": 2 },
        "name": { "type": "string", "minLength": 2, "maxLength": 30 },
        "email": { "type": "string", "format": "email" },
        "slackUserId": { "type": "string" },
        "active": { "type": "integer", "enum": [0, 1] }
      },
      "required": ["id", "mbId", "initials", "name", "email", "slackUserId", "active"]
    }
  }
}
```

```
},  
  "required": ["data"]  
}
```